

Xinu (RAZ 拡張版) ユーザズマニュアル

XFS ファイルシステム・cc C コンパイラ・abclc アクタ言語・wm ウィンドウマネージャ

本マニュアルは、Embedded Xinu をベースに、階層型ファイルシステム **XFS** (RAM ディスク上)、組込みのちっちゃな C コンパイラ **cc** とバイトコード VM **a.out**、アクタ指向小型言語 **ABCL/c+** とその C トランスレータ **abclc**、ウィンドウマネージャ **wm** (ARM/QEMU 上のフレームバッファ GUI)、Unix 風シェル拡張 (**ls/cd/cat/make/edit** など) を組み込んだ拡張版 Xinu の使い方をまとめたものです。本家 Embedded Xinu の機能 (スレッド・セマフォ・ネットワーク等) はそのまま使えます。

目次

1	はじめに	1
2	起動方法	1
2.1	ビルド	1
2.2	QEMU で実行	2
2.3	起動シーケンス	2
2.4	実機 (Raspberry Pi 3 B+) での実行	2
2.5	WiFi への接続 (Pi 3 B+)	4
2.6	複数 Xinu でのメッシュネットワーク (MANET ad-hoc)	4
3	シェルの使い方	5
3.1	基本	5
3.2	コマンド一覧 (本拡張版で追加・拡張されたもの)	5
4	XFS — ファイルシステム	6
5	cc / run — C コンパイラとランチャ	7
5.1	サポートする C のサブセット	7
5.2	組み込み関数 (Built-in)	7
5.3	a.out 形式とシェルでの使い方	7
6	abclc — ABCL/c+ アクタ言語	8
7	make — 簡易ビルドシステム	8
8	edit — 組み込みエディタ	9

9	wm — ウィンドウマネージャ	10
10	サンプルプログラム詳説	10
10.1	hello.c / sum.c	10
10.2	PingPong.abcl — アクタ間メッセージング	11
10.3	RotLines.abcl — ABCL + WM のアニメーション	11
10.4	rotlines (シェルコマンド) と apps/abcl_program.c	12
11	開発ワークフロー	12
12	トラブルシューティング	12
付録 A.	ファイルマップ	13
付録 B.	既存サンプル一覧	13

1 はじめに

Xinu は Douglas Comer が教育用に開発した小型 UNIX 風 OS です。本拡張版はこれに「OS 上で C/ABCL ソースを編集・コンパイル・実行できる自己完結したワークベンチ」を載せたもので、ホスト側のクロスコンパイラに頼らずとも、QEMU の中だけでプログラム開発のループを完結できます。

プラットフォーム名	用途	特徴
arm-qemu	QEMU -M versatilepb	UART シェル + フレームバッファ + WM 自動起動
arm-rpi	Raspberry Pi 実機	UART / HDMI / USB キーボード

XFS / cc / abclc / make / edit はどのプラットフォームでも同じように使えます。WM 関連 (rotlines, wm_line など) は arm-qemu でのみ有効です。

2 起動方法

2.1 ビルド

```
cd compile
make PLATFORM=arm-qemu          # 既定。arm-rpi も可
```

成功すると compile/xinu.boot が生成されます。

2.2 QEMU で実行

ホストが macOS/Linux で qemu-system-arm がインストール済みであれば、用意してあるラップスクリプトが便利です。

コンソール専用モード (フレームバッファなし):

```
./compile/run-console.sh
```

UART シェル (xsh\$) が端末に多重化されます。終了は **Ctrl-A** → x。環境変数: XINU_MEM (既定 128M)、XINU_PLATFORM (既定 arm-qemu)。

ウィンドウモード (WM デモ込み):

```
./compile/run-window.sh
```

LCD フレームバッファが macOS の Cocoa ウィンドウに表示されます。UART シェルは起動した端末で動きます (-serial mon:stdio で QEMU モニタも同居)。終了はウィンドウを閉じるか端末で **Ctrl-A** → x。ウィンドウは角をドラッグで最大 4 倍まで滑らかに拡大可能 (zoom-to-fit)。

2.3 起動シーケンス

system/main.c の main() スレッドが以下を順に行います。

1. OS バージョン・メモリレイアウトを表示
2. xfsBootstrap() で RAMDISK0 を XFS でフォーマット & / にマウント
3. /home/hello.c, sum.c, PingPong.abcl, RotLines.abcl をシード書き込み
4. ネットワークデバイスを open (NETHER 有効時)
5. CONSOLE/TTY1 を open
6. arm-qemu のときだけ wm_main スレッドを create()
7. シェルスレッドを CONSOLE/TTY1 ごとに起動

ブート完了後、シェルプロンプトに xsh\$ と表示されればコマンド入力可能です。

2.4 実機 (Raspberry Pi 3 B+) での実行

QEMU だけでなく、実機の Raspberry Pi 3 B+ (BCM2837) でもこのワークベンチを動かせます。実機向けは専用プラットフォーム arm-rpi3 でビルドし、microSD カードにカーネルを焼いて起動します。

実機向けビルド:

```
cd compile
make PLATFORM=arm-rpi3          # -> compile/xinu.boot (~305 KB)
```

共有ヘッダ (例: include/thread.h) を変更した場合、make clean では libxc が再ビルドされません。printf 系がおかしくなったら強制再ビルド: `rm -f lib/libxc/*.o lib/libxc.a && make PLATFORM=arm-rpi3`。

SD カードに必要なもの。Pi 3 のファームウェアは SD の FAT32 パーティションから起動します。

Broadcom ファームウェア (bootcode.bin, start.elf, fixup.dat — Raspberry Pi OS の boot パーティションからコピー。本リポジトリには含まれません)、kernel.img (ビルドした xinu.boot をリネーム)、config.txt (compile/sdcard-rpi3/config.txt) を置きます。主な config.txt の設定:

行	意味
arm_64bit=0	32-bit (AArch32) で起動
kernel=kernel.img	起動カーネル名
kernel_address=0x8000	ロードアドレス (0x8000)
enable_uart=1	UART を有効化
dtoverlay=disable-bt	PL011 (UART0) を GPIO14/15 に配線 (Bluetooth 無効化)
init_uart_clock=48000000	PL011 基準クロック 48 MHz (115200 8N1 用)
hdmi_force_hotplug=1	モニタ未検出でも HDMI 出力を強制

SD カードの作り方 (方法 A — FAT32 で新規作成、推奨):

```
diskutil list                                # microSD の diskN を確認 (内蔵 disk0 と間違えない!)
diskutil eraseDisk FAT32 XINU MBRFormat /dev/diskN # カードを初期化 (中身は消える)
cp /Volumes/bootfs/{bootcode.bin,start.elf,fixup.dat} /Volumes/XINU/
cp compile/sdcard-rpi3/config.txt /Volumes/XINU/
cp compile/xinu.boot /Volumes/XINU/kernel.img
diskutil eject /Volumes/XINU
```

方法 B — 既存の Raspberry Pi OS カードを再利用 (フォーマット不要): その boot パーティション (firmware は既に在る) に kernel.img と本リポジトリの config.txt だけをコピーします (config.txt が kernel=kernel.img を指定)。

シリアルコンソールの配線。 3.3V の USB-TTL シリアル変換を使います (アダプタの 5V は接続しない)。GND → pin 6、アダプタ RX → pin 8 (GPIO14/TXD)、アダプタ TX → pin 10 (GPIO15/RXD)。115200 8N1 で開く: screen /dev/tty.usbserial-XXXX 115200。

起動。 SD を Pi 3 に挿し電源 ON。十数~40 秒ほどでシリアルに xsh\$ プロンプトが出ます (gwm 入りビルドでは HDMI デスクトップも表示)。あとは QEMU と同じく cc / abcl / make / run が使えます。Pi 3 B+ ポートの HDMI デスクトップ (gwm) ・WiFi ・HTTP サーバ等の操作は README.md と docs/USERS-GUIDE-JA.md を参照してください。

SD 交換なしのカーネル更新 (/kexec)。 /kexec 対応ビルドが既に動いていれば、ネットワーク越しにカーネルを差し替えられます:

```
curl --data-binary @compile/xinu.boot "http://192.168.3.50:8080/upload?dst=xinu.boot"
curl -X POST http://192.168.3.50:8080/kexec
sleep 25
curl http://192.168.3.50:8080/api/mmu          # 新カーネルが起動すれば応答する
```

これは揮発的で、電源を入れ直すと SD から再起動します。

2.5 WiFi への接続 (Pi 3 B+)

arm-rpi3 ビルドには BCM43455 WiFi ドライバ (Arasan SDIO 経由) が入っています。起動時に自動接続はしません — 毎回 `wifi on` してください。xsh\$ シェルから:

コマンド	説明
<code>wifi status</code>	現在の接続 (SSID + IP/GW) を表示
<code>wifi on</code>	対話: スキャン (30 秒、無線が一瞬切れる) → AP 一覧 → 番号で選択 → パスワード (空 = オープン) → join + DHCP。成功後 ARP/ICMP 応答を開始し ping 可能に
<code>wifi on <ssid> [pass]</code>	非対話 (スキャン/プロンプトなし)。/shell?cmd= リモートログイン (stdin なし) 用。pass 省略/空 = オープン
<code>wifi off</code>	切断 (BSS ダウン)
<code>wifi-invest</code>	「接続済みなのに ping 不通」のとき、応答スレッド再起動・gratuitous ARP・ゲートウェイ ping を通るまで自動リトライ

```
xsh$ wifi on
Scanning for access points (~30s; the radio drops briefly)...
Available networks:
  1: MyHome-5G
  2: Buffalo-G-56A2
Select AP number (0 = cancel): 1
Password for "MyHome-5G" (blank = open): *****
Connecting to "MyHome-5G"...
WiFi: connected to "MyHome-5G"
  ip 192.168.3.52 gw 192.168.3.1
```

スキャンは一瞬 WiFi を切ります (全チャンネル再同調のため)。再起動後は自動再接続しないので、起動のたびに `wifi on` してください。有線側の HTTP からでも操作できます: `/api/wifi/scan`、`/api/wifi/join?ssid=S&pass=P`、`/api/wifi/dhcp`、`/api/wifi/ping?ip=A.B.C.D`。

2.6 複数 Xinu でのメッシュネットワーク (MANET ad-hoc)

複数の Xinu ボードを **アクセスポイント無し** で直接つなぎ、ピアツーピアのメッシュを組めます。802.11 の IBSS (ad-hoc) モード + オンデマンドの AODV ルーティングを使います (ドローン HIL デモの Pi 4 + Pi 3 ノードと同じ仕組み)。各ノードは同じ ad-hoc セルに、別々のノード番号で参加します:

```
xsh$ wifi adhoc <cell-ssid> [channel] [node]
```

- `<cell-ssid>` — ad-hoc セル名。全ノードで同じ名前・同じチャンネルにすること。
- `[channel]` — 802.11 チャンネル (既定 6)。
- `[node]` — 自分のノード番号 (既定 2)。静的 IP `10.0.0.<node>/24` を得ます。ノードごとに別の番号を与えること。

成功すると `ad-hoc up, ip 10.0.0.<node> (responder running)` と表示され、`10.0.0.<node>` で ping に応答します。例 — チャンネル 6 の 3 ノードセル:

```
xsh$ wifi adhoc mesh1 6 1      # ノード 1 -> 10.0.0.1
xsh$ wifi adhoc mesh1 6 2      # ノード 2 -> 10.0.0.2
xsh$ wifi adhoc mesh1 6 3      # ノード 3 -> 10.0.0.3
```

電波の届く範囲のノードは `10.0.0.x` で直接通信できます (同じ IBSS セルに `10.0.0.x/24` で参加した Mac から ping `10.0.0.1` 等も可能)。

マルチホップ (AODV)。宛先が直接届かないとき、最小 AODV モジュール (M13) がオンデマンドで経路探索します: 発信ノードが RREQ をブロードキャスト (UDP port 654) し、範囲内の各ノードが中継して逆経路を記録、宛先が RREP を返して順経路 (route to `10.0.0.x` via `10.0.0.y` (k hop)) を確立します。各ノードは応答スレッドの一部として AODV 制御 (中継 + 返信) を自動で回すので、直接届かないピア宛のトラフィックも中間ノードが転送します (経路表は最大 16 エントリ)。有線 HTTP からノードを ad-hoc に上げることもできます: `curl "http://<node-ip>:8080/wifi-adhoc?ssid=mesh1&ch=6&n=2"`。

IBSS/ad-hoc はインフラ (wifi on) モードとは独立です (AP に繋がっている場合は先に wifi off)。ad-hoc も再起動後は復元しないので、各ノードで wifi adhoc を再実行してください。

3 シェルの使い方

3.1 基本

```
コマンド名 [引数...]    [< 入力ダイレクト] [> 出力ダイレクト] [&]
```

& でバックグラウンド起動、< > で入出力デバイスへリダイレクト。補完・履歴はありません。

3.2 コマンド一覧 (本拡張版で追加・拡張されたもの)

コマンド	使い方	説明
pwd	pwd	カレントディレクトリ表示
cd	cd [PATH]	ディレクトリ変更 (省略時は /)
ls	ls [-l] [PATH]	一覧。-l で type/size 表示
cat	cat FILE...	ファイル内容表示
mkdir	mkdir DIR...	ディレクトリ作成
rmdir	rmdir DIR...	空ディレクトリ削除
touch	touch FILE...	空ファイル作成 / mtime 更新
rm	rm FILE...	ファイル削除
cp	cp SRC DST	コピー (DST は truncate)
mv	mv SRC DST	リネーム/移動
write	write FILE TEXT...	引数を空白区切りで連結し改行付きで書き込み
edit	edit FILE	組込み Emacs 風エディタ
mkfs	mkfs DEVICE [VOL]	ブロックデバイスを XFS でフォーマット

コマンド	使い方	説明
mount	mount DEV MNT	XFS をマウント
umount	umount MNT	XFS をアンマウント
cc	cc SRC [-o OUT]	C ソースを a.out にコンパイル
abclc	abclc SRC.abcl [-o OUT]	ABCL → C → a.out 一気通貫
make	make [TARGET...]	カレントの Makefile を実行
run	run PATH	a.out を実行
rotlines	rotlines [F] [DEG/F]	WM 上に回転線を描く
clear	clear	画面クリア
sleep	sleep N	N ms スリープ

暗黙の **run**。シェルが知らないコマンド名を打つと、最後に `aoutRun(<その名前>)` を試します (`shell/shell.c:332` 付近)。つまり `cc hello.c -o hello` のあとは `xsh$ hello` でファイルパス相当の名前から直接プログラムを起動できます。

4 XFS — ファイルシステム

`include/xfs.h` で定義される 4 KB ブロックの階層型 FS で、起動時に 16 MB の RAM ディスク (`RAMDISK0`) 上に作られ / にマウントされます。

項目	値
ブロックサイズ	4096 B
最大ファイル名長	56 B
inode サイズ	128 B
直接ブロック	12 (= 48 KB)
一段間接	1024 ブロック (≒ 4 MB)
二段間接	あり
マジック	XFS! (0x58465321)

ディレクトリは inode の中身が `struct xdirent[]` (1 エントリ 64 B) で表現され、`xfsReaddir()` で順次読み出します。`xfsChdir()/xfsGetcwd()` はスレッドごとの絶対パス CWD を保持します (シェル A で `cd /home` してもシェル B には影響しません)。`xfsBootstrap()` は `RAMDISK0` 先頭 64 KB に XFS マジックがあるか調べ、なければ `xfsMkfs` で初期化し / にマウントします。

```
int fd = xfsOpen("/home/foo", XFS_0_RDWR | XFS_0_CREAT | XFS_0_TRUNC);
xfsWrite(fd, buf, n); xfsRead(fd, buf, n);
xfsSeek(fd, 0, XFS_SEEK_SET); xfsClose(fd);
xfsMkdir("/home/sub"); xfsRename("/home/old", "/home/new"); xfsUnlink("/home/foo");
```

`RAMDISK0` は名前のとおり RAM です。**QEMU** を終了するとファイルは消えます。永続的に残したいソースはホストでバックアップしてください。

5 cc / run — C コンパイラとランチャ

5.1 サポートする C のサブセット

system/cc.c 冒頭に明記: 1 ソースファイル、int main(void) 1 個 + 引数なし補助関数。宣言は int x; / int x = expr; のみ。文は if/else, while, for, return, ブロック, 式文。式は整数・文字列・識別子・+ - * / %・比較・! && ||・単項 -・括弧。組込み関数のみ呼出可。#include は受理して無視。**未サポート**: 構造体・ポインタ・配列・浮動小数・複数ファイル。

5.2 組込み関数 (Built-in)

BI	関数	用途
0	printf(fmt, ...)	文字列・整数フォーマット
1	puts(s)	文字列 + 改行
2	putchar(c)	1 文字出力
3	getchar()	1 文字入力
4	exit(code)	終了
5	rgb(r,g,b)	BGR565 16bit カラー値
6	wm_line(idx,x1,y1,x2,y2,color)	WM のユーザ線スロット (0..7) に登録
7	wm_render(on)	ユーザ線レンダの有効/無効
8	wm_clear()	ユーザ線クリア
9	sleep_ms(ms)	スリープ
10/11	isin(deg)/icos(deg)	Q12 固定小数 (4096 = 1.0)
12/13	screen_w()/screen_h()	画面サイズ取得

5.3 a.out 形式とシェルでの使い方

include/aout.h の struct aout_header:

```
+-----+
| magic = "XAOU"          |
| version = 1             |
| code_size / const_size  |
| entry / nlocals         |
+-----+
| バイトコード本体        |
| 文字列定数プール (零終端) |
+-----+
```

VM はスタック型 (256 entries) で、OP_PUSH_I32 OP_LOAD_L0C OP_ADD OP_JZ OP_CALL_BI などの 1 バイトオペコードを持ちます (system/aout.c の aoutRun())。

```
xsh$ cc /home/hello.c -o /home/hello
xsh$ run /home/hello
```



```
hello...
xsh$ /home/hello          # 暗黙 run でも OK
```

6 abclc — ABCL/c+ アクタ言語

ABCL/c+ は「クラス + メッセージ送信」によるアクタ指向の小型言語で、本実装では **abclc** が **C** へ変換し、その **C** を **cc** がバイトコード化する 2 段階構成です。

```
class ClassName {
  var field1;
  method methodName(p1, p2) {
    field1 = p1 + 1;
    send other.methodName(arg1, arg2);
  }
}
main {
  new ClassName a; new ClassName b;
  send a.methodName(b, 10);
}
```

暗黙の識別子 **self** (現在のアクタ ID)・**sender** (送信元 ID)。1 メソッドあたり **send** は最大 1 回 (末尾呼び出し型) なので、1 アクタは 1 メッセージずつ逐次処理されます。値はすべて **int** 相当。制限: 4 クラス、1 クラス 16 メソッド、8 フィールド、8 引数。

```
xsh$ cd /home/abclcp/abclc
xsh$ abclc PingPong.abcl
abclc: translated PingPong.abcl -> PingPong.c
abclc: compiled PingPong.c -> PingPong
xsh$ run PingPong
```

-o NAME を付けると **NAME.c** (中間 C) と **NAME (a.out)** の両方が作られます。abclc が出力する C は cc のビルトインと少数のスケジューリング用ヘルパだけを使い、1 メッセージ = 1 タイムスライスで実行され、**send** 後はキューにぶら下がります。

7 make — 簡易ビルドシステム

shell/xsh_make.c: CWD に Makefile がなければ *.c と *.abcl を走査して自動生成し、**TARGETS = ...** で指定された (またはルール宣言順の) 各ターゲットをビルド。**.c** → **cc SRC -o TARGET**、**.abcl** → **abclc SRC -o TARGET**。

```
# コメント
TARGETS = hello sum PingPong RotLines
hello:   hello.c
sum:     sum.c
```

```
PingPong: PingPong.abcl
RotLines: RotLines.abcl
```

明示ルールが無いターゲットは TARGET.c → TARGET.abcl の順に存在チェックして自動推論します。

```
xsh$ cd /home
xsh$ make
make: generated Makefile (2 .c, 0 .abcl)
    cc hello.c -o hello
    cc sum.c -o sum
make: built 2 target(s)
xsh$ ./hello
hello...
```

8 edit — 組み込みエディタ

shell/xsh_edit.c の Emacs 風モードレスエディタ (最大 400 行 × 256 文字)。edit /home/hello.c。存在しないパスは新規バッファで開きます。

キー	動作
C-f / C-b	1 文字 進む / 戻る
C-n / C-p	1 行 下 / 上
C-a / C-e	行頭 / 行末
←↑→↓	矢印 (ESC[シーケンス)
C-d	カーソル位置を削除
Backspace	1 文字戻して削除
Enter	改行挿入
Tab	空白 2 個
C-k	行末まで kill
C-l	再描画
C-x C-s	保存
C-x C-c	保存して終了
C-g	保存せず終了

ステータス行に L 行:C 列 status が表示されます。

9 wm — ウィンドウマネージャ

apps/wm.c は arm-qemu の VersatilePB 上で PL110 LCD コントローラ (0x10120000, 16bpp BGR565) と PL050 PS/2 マウス (0x10007000) を直接叩いて、1024×1024 仮想画面 (実画面 640×480) のデスクトップ風 GUI を表示します。run-window.sh で起動するとブート時に自動で wm_main が立ち上がります。提供するもの: ステータスバー、ドラッグ可能なデモウィンドウ 3 枚、マウスカーソル、ユーザ用描画フック wm_set_user_render(...), 直接描画 API (put_pixel_pub / fill_rect_pub / rect_outline_pub / draw_string_pub / wm_draw_line)。

C プログラム / ABCL からは a.out のビルトイン経由で `rgb(r,g,b)・wm_render(1)・wm_line(idx,...)・wm_clear()・screen_w()/screen_h()` を使うのが基本です (RotLines.abcl 参照)。シェルコマンド `rotlines` はもう少し低レベルで、`wm_set_user_render` で毎フレームの描画関数を直接 WM に登録します。

10 サンプルプログラム詳説

ブート時に `system/main.c` が以下 4 本を XFS にシードします。

パス	種類	主役
/home/hello.c	C	printf
/home/sum.c	C	for ループ
.../PingPong.abcl	ABCL	アクタ間メッセージ
.../RotLines.abcl	ABCL + WM	アニメーション

10.1 hello.c / sum.c

```
int main(void) { printf("hello...\n"); return 0; }
```

`#include` は受理して無視、`printf` はビルトイン 0、戻り値は `aoutRun()` の戻り値になります。簡略バイトコード:

```
ENTER 0 ; main は局所なし
PUSH_STR offset_of("hello...\n")
CALL_BI 0, 1 ; printf, 引数 1
PUSH_I32 0
RET
```

`sum.c` は for ループの例 (`cc` は `for(init;cond;step)` を `init; while(cond){body; step;}` に展開。+= ++ -- は未サポート):

```
xsh$ cc /home/sum.c -o /home/sum
xsh$ run /home/sum
sum 1..10 = 55
```

10.2 PingPong.abcl — アクタ間メッセージング

```
class Player {
  var hits;
  method bounce(other, n) {
    if (n > 0) {
      printf("Player %d: tick (n=%d) hits=%d\n", self, n, hits);
      hits = hits + 1;
    }
  }
}
```

```

        send other.bounce(self, n - 1);
    }
}
main { new Player p1; new Player p2; send p1.bounce(p2, 6); }

```

`self` はメソッド内の現在アクタ ID (生成順 0,1,...)、`send` は**非同期**、`n-1` が 0 で `if` を抜けると `send` が出ず連鎖は自然停止します。

```

Player 0: tick (n=6) hits=0
Player 1: tick (n=5) hits=0
Player 0: tick (n=4) hits=1
...
Player 1: tick (n=1) hits=2

```

10.3 RotLines.abcl — ABCL + WM のアニメーション

```

class Rotator {
  var angle;
  method spin(n) {
    if (n > 0) {
      wm_line(0, 512, 384,
              512 + icos(angle) * 320 / 4096,
              384 + isin(angle) * 320 / 4096, rgb(255, 80, 80));
      // スロット 1..3 を +90/+180/+270 度で同様に
      sleep_ms(16);
      angle = angle + 3;
      send self.spin(n - 1);
    } else { wm_render(0); }
  }
  method start(n) { wm_render(1); send self.spin(n); }
}
main { new Rotator r; send r.start(360); }

```

`icos/isin` は Q12 固定小数を返すので半径を掛けたら 4096 で割る。`sleep_ms(16)` でだいたい 60 fps。`self` への再帰送信でアニメーションループを作り、終了条件 (`n == 0`) で `wm_render(0)` を呼んで線レイヤを消します。

10.4 rotlines (シェルコマンド) と apps/abcl_program.c

`shell/xsh_rotlines.c` は `rot_render()` を `wm_set_user_render()` で WM に渡し、メインスレッドは角度を進めながら `sleep(16)` を繰り返すだけ。描画は (スロット型の `wm_line` ではなく) 毎フレーム関数の `wm_draw_line`。`apps/abcl_program.c` は「`abclc` が ABCL から**生成する側**の C」の見本で、有限バッファ問題 (Buffer/Producer/Consumer/Controller クラス、アクタ毎にメールボックス + スレッド、`dispatch_*` メガディスパッチ) を実装しています。ABCL の `class` が C の `dispatch_<Class>` に、

field が enum インデックスに、send が enqueue に対応する様子を読み取れます。

11 開発ワークフロー

```
$ ./compile/run-window.sh
xsh$ cd /home && make && ./hello && ./sum
xsh$ edit /home/myprog.c          # C-x C-s 保存、C-x C-c 終了
xsh$ cc myprog.c -o myprog && ./myprog
xsh$ cd /home/abclcp/abclc
xsh$ edit MyActor.abcl && abclc MyActor.abcl && ./MyActor
xsh$ rotlines
```

ヒント: 1 ターゲットだけなら make MyActor、全部作り直すなら rm Makefile && make。XFS の inode は 32768 までなので大量作成は注意。

12 トラブルシューティング

症状	原因 / 対処
qemu-system-arm not found	brew install qemu / apt install qemu-system-arm
ビルドが古いまま起動する	compile/ で make clean && make PLATFORM=arm-qemu
シェルプロンプトが出ない	Ctrl-A → c で QEMU モニタ、info registers
cc: line N: ... エラー	サポート外構文 (構造体・ポインタ・配列・浮動小数 等)
abclc: translation failed	1 メソッドに send 2 個、var/メソッド数オーバ 等
command not found だが実体あり	暗黙 run はパス 1 トークンのみ。引数は run PATH ARGS...
WM ウィンドウが出ない	arm-qemu ビルドか? run-window.sh を使ったか?
ファイルが消えた	RAMDISK は揮発性。ホスト側にバックアップを
パスが長すぎる	XFS_PATH_MAX = 256。長すぎるパスは切り詰め
画面が崩れた	エディタ中なら C-l、シェルなら clear

付録 A. ファイルマップ

領域	主要ファイル
起動	system/main.c, system/initialize.c, compile/run-*.sh
XFS	include/xfs.h, system/xfs.c, device/ramdisk/
cc	include/aout.h, system/cc.c, system/aout.c, shell/xsh_cc.c
abclc	include/abcl.h, system/abcl.c, shell/xsh_abcl.c
make/edit	shell/xsh_make.c, shell/xsh_edit.c
WM	apps/wm.c, apps/abcl_xinu_gui.c, shell/xsh_rotlines.c
プラットフォーム	compile/platforms/arm-qemu/, compile/platforms/arm-rpi/

付録 B. 既存サンプル一覧 (FS シード後)

```
/home/  
├── hello.c  
├── sum.c  
└── abclcp/  
    ├── abclc/  
    │   ├── PingPong.abcl  
    │   └── RotLines.abcl
```

`cd /home && make` で `hello`, `sum` が、`cd /home/abclcp/abclc && make` で `PingPong`, `RotLines` がそれぞれ `a.out` として生成されます。さらに深く知るには `system/cc.c` と `system/aout.c` (バイトコード VM)、`system/abclc.c` (ABCL 翻訳) を読むのが近道です。